

Université de Nouakchott

Faculté des Sciences et Techniques

Module : Recherche Opérationnelle (Licence)

Rapport de Projets

Recherche Opérationnelle 2026

Étudiant : Daouda Abou Dia

Responsable : Dr. EL BENANY Med MAhמוד

Année : 2025–2026

Sujet	Catégorie
1.4 – Méthode du Simplexe	Classiques et fondamentaux
2.1 – Problème du Voyageur (TSP)	Logistique et supply chain
6.3 – Routage des camions d'eau	Problèmes réels (RIM)

Outils : Python (PuLP, NumPy, Matplotlib) | GitHub :
https://github.com/ArdoDia04/R0_2026

Table des matières

1	Sujet 1 : Méthode du Simplexe	2
1.1	Problème	2
1.2	Modélisation	2
1.3	Résolution par le Simplexe	2
1.3.1	Forme standard	2
1.3.2	Itérations du Simplexe	2
1.4	Solution optimale	3
1.5	Implémentation Python	3
1.6	Visualisation graphique	4
2	Sujet 2 : Problème du Voyageur (TSP)	5
2.1	Problème	5
2.2	Modélisation mathématique	5
2.3	Instance utilisée	5
2.4	Méthodes de résolution	5
2.5	Résultats	5
2.6	Implémentation Python	6
2.7	Visualisation	6
3	Sujet 3 : Routage des camions d'eau – Nouakchott (RIM)	7
3.1	Contexte	7
3.2	Données	7
3.3	Modélisation	7
3.4	Résolution par heuristique greedy	7
3.5	Résultats	8
3.6	Implémentation Python	8
3.7	Visualisation	9
4	Conclusion générale	10

Sujet 1 : Méthode du Simplexe

Problème

On cherche à maximiser le profit d'une production à deux produits sous contrainte de ressources :

$$\text{Max } z = 3x_1 + 4x_2 \quad (1)$$

$$\text{s.t. } x_1 + 2x_2 \leq 8 \quad (2)$$

$$3x_1 + x_2 \leq 9 \quad (3)$$

$$x_1, x_2 \geq 0 \quad (4)$$

Modélisation

Les variables de décision sont :

- x_1 : quantité produite du produit 1
- x_2 : quantité produite du produit 2

L'objectif est de maximiser le profit total $z = 3x_1 + 4x_2$ sous contrainte des ressources disponibles. Chaque contrainte représente une limite de capacité (heures machine, matières premières, etc.).

Résolution par le Simplexe

1.3.1 Forme standard

On introduit les variables d'écart $s_1, s_2 \geq 0$ pour transformer les inégalités en égalités :

$$x_1 + 2x_2 + s_1 = 8$$

$$3x_1 + x_2 + s_2 = 9$$

$$z - 3x_1 - 4x_2 = 0$$

1.3.2 Itérations du Simplexe

Tableau initial :

Base	x_1	x_2	s_1	s_2	b
s_1	1	2	1	0	8
s_2	3	1	0	1	9
z	-3	-4	0	0	0

TABLE 1 – Tableau initial — colonne pivot : x_2 (plus négatif)

Ratio test : $8/2 = 4$ et $9/1 = 9 \Rightarrow$ ligne pivot : s_1 . On divise la ligne s_1 par 2.

Tableau après 1^{ère} itération :

Base	x_1	x_2	s_1	s_2	b
x_2	1/2	1	1/2	0	4
s_2	5/2	0	-1/2	1	5
z	-1	0	2	0	16

TABLE 2 – Après 1^{ère} itération — colonne pivot : x_1

Ratio test : $4/(1/2) = 8$ et $5/(5/2) = 2 \Rightarrow$ ligne pivot : s_2 .

Tableau final :

Base	x_1	x_2	s_1	s_2	b
x_2	0	1	3/5	-1/5	3
x_1	1	0	-1/5	2/5	2
z	0	0	9/5	2/5	18

TABLE 3 – Tableau final — aucune valeur négative dans la ligne z

Solution optimale

$$x_1^* = 2, \quad x_2^* = 3, \quad z^* = 3(2) + 4(3) = \mathbf{18}$$

Il faut produire 2 unités du produit 1 et 3 unités du produit 2 pour obtenir un profit maximum de 18.

Implémentation Python

```

1 from pulp import *
2
3 prob = LpProblem("Simplexe", LpMaximize)
4
5 x1 = LpVariable("x1", lowBound=0)
6 x2 = LpVariable("x2", lowBound=0)
7
8 prob += 3*x1 + 4*x2          # Fonction objectif
9 prob += x1 + 2*x2 <= 8      # Contrainte 1
10 prob += 3*x1 + x2 <= 9     # Contrainte 2
11
12 prob.solve(PULP_CBC_CMD(msg=0))
13
14 print(f"x1 = {value(x1)}")   # x1 = 2.0
15 print(f"x2 = {value(x2)}")   # x2 = 3.0
16 print(f"z = {value(prob.objective)}") # z = 18.0

```

Listing 1 – Résolution du PL avec PuLP

Visualisation graphique

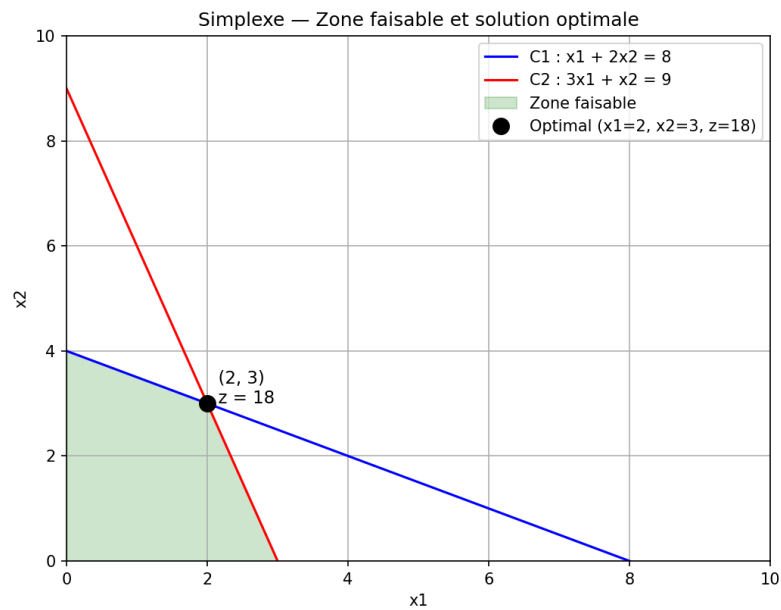


FIGURE 1 – Zone faisable et solution optimale du Simplexe

La zone verte représente l'ensemble des solutions réalisables. La solution optimale se trouve au sommet $(2, 3)$, point d'intersection des deux contraintes actives.

Sujet 2 : Problème du Voyageur (TSP)

Problème

Trouver le cycle hamiltonien de longueur minimale passant par n villes une seule fois et revenant à l'origine. C'est un problème NP-difficile : le nombre de solutions possibles est $(n - 1)!$.

Modélisation mathématique

Soit $x_{ij} \in \{0, 1\}$ valant 1 si l'arc (i, j) est emprunté, et c_{ij} la distance entre les villes i et j .

$$\text{Min } \sum_i \sum_{j \neq i} c_{ij} x_{ij} \quad (5)$$

$$\text{s.t. } \sum_{j \neq i} x_{ij} = 1 \quad \forall i \quad (\text{quitter chaque ville une fois}) \quad (6)$$

$$\sum_{i \neq j} x_{ij} = 1 \quad \forall j \quad (\text{entrer dans chaque ville une fois}) \quad (7)$$

$$x_{ij} \in \{0, 1\} \quad (8)$$

Instance utilisée

5 villes avec coordonnées : A(0,0), B(2,4), C(5,2), D(6,5), E(3,1).

Méthodes de résolution

Heuristique greedy (plus proche voisin) : À chaque étape, on se déplace vers la ville non visitée la plus proche. Rapide mais non garanti optimal.

Brute-force (exact) : On teste toutes les $(n - 1)! = 24$ permutations et on garde la plus courte. Exact mais exponentiel.

Résultats

TABLE 4 – Comparaison des méthodes

Méthode	Chemin	Distance
Greedy	A → B → D → C → E → A	17.15
Exact	A → B → D → C → E → A	17.15

Sur cette instance, les deux méthodes coïncident. Sur de grandes instances, le greedy est moins précis mais beaucoup plus rapide.

Implémentation Python

```

1 import numpy as np
2 import itertools
3
4 villes = ['A', 'B', 'C', 'D', 'E']
5 coords = {'A':(0,0), 'B':(2,4), 'C':(5,2), 'D':(6,5), 'E':(3,1)}
6
7 def distance(v1, v2):
8     x1,y1 = coords[v1]; x2,y2 = coords[v2]
9     return round(np.sqrt((x2-x1)**2+(y2-y1)**2), 2)
10
11 def greedy_tsp(depart):
12     non_visites = list(villes)
13     non_visites.remove(depart)
14     chemin = [depart]; current = depart; total = 0
15     while non_visites:
16         voisin = min(non_visites, key=lambda v: distance(current,v)
17                     )
18         total += distance(current, voisin)
19         chemin.append(voisin)
20         non_visites.remove(voisin)
21         current = voisin
22     total += distance(current, depart)
23     chemin.append(depart)
24     return chemin, round(total, 2)
25
26 chemin, dist = greedy_tsp('A')
27 print(f"Chemin : {' -> '.join(chemin)}")
28 print(f"Distance : {dist}")

```

Listing 2 – TSP – Heuristique du plus proche voisin

Visualisation

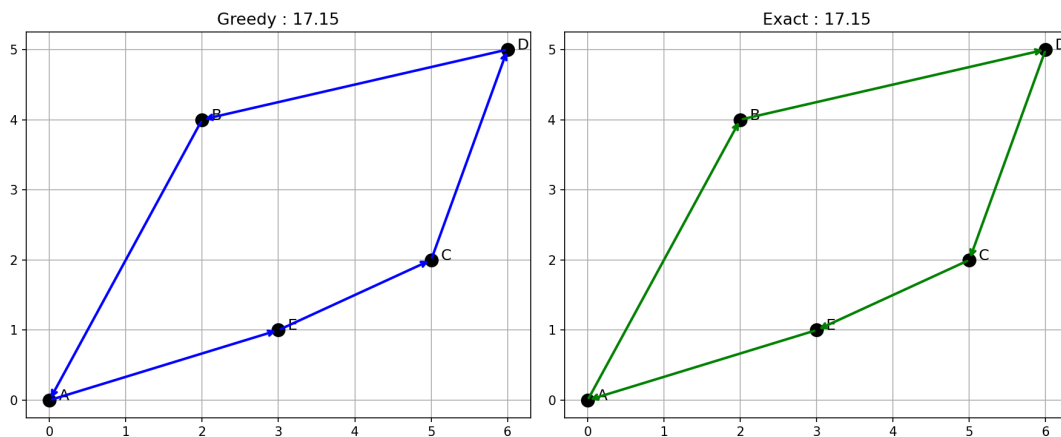


FIGURE 2 – Comparaison Greedy vs Exact pour le TSP (5 villes)

Sujet 3 : Routage des camions d'eau – Nouakchott (RIM)

Contexte

Dans les quartiers périphériques de Nouakchott, la distribution d'eau potable repose sur des camions-citernes. L'objectif est de planifier les tournées pour minimiser la distance parcourue tout en respectant la capacité du véhicule.

Données

TABLE 5 – Demandes des quartiers

Quartier	Demande (L)
Arafat	300
Sebkha	200
El Mina	250
Dar Naim	150
Teyarett	180
Toujounine	220
Total	1300

Capacité du camion : **1000 L**. La demande totale (1300 L) dépasse la capacité, donc au moins 2 tournées sont nécessaires.

Modélisation

Le VRP (Vehicle Routing Problem) est une généralisation du TSP avec contrainte de capacité. On cherche à minimiser la distance totale des tournées tout en respectant :

- Chaque client est visité exactement une fois
- La charge de chaque tournée ne dépasse pas la capacité du camion
- Chaque tournée commence et termine au dépôt

Résolution par heuristique greedy

À chaque étape, on choisit le client non visité le plus proche qui ne dépasse pas la capacité restante. Quand plus aucun client n'est accessible, on retourne au dépôt et on commence une nouvelle tournée.

Résultats

TABLE 6 – Tournées obtenues

Tournée	Chemin	Charge (L)
1	Depot → Sebkha → El Mina → Toujounine → Dar Naim → Teyarett → Depot	1000
2	Depot → Arafat → Depot	300
Total		1300

La tournée 1 utilise exactement la capacité maximale du camion. Arafat (300 L) est livré seul lors de la tournée 2 car il ne restait plus de capacité.

Implémentation Python

```

1 import numpy as np
2
3 points = {
4     'Depot':(0,0), 'Arafat':(3,4), 'Sebkha':(-2,3),
5     'El Mina':(-4,1), 'Dar Naim':(2,-3),
6     'Teyarett':(4,-1), 'Toujounine':(-1,-4)
7 }
8 demandes = {
9     'Arafat':300, 'Sebkha':200, 'El Mina':250,
10    'Dar Naim':150, 'Teyarett':180, 'Toujounine':220
11 }
12 CAPACITE = 1000
13
14 def distance(a, b):
15     x1,y1=points[a]; x2,y2=points[b]
16     return round(np.sqrt((x2-x1)**2+(y2-y1)**2), 2)
17
18 def vrp_greedy():
19     non_visites = list(demandes.keys())
20     tournées = []
21     while non_visites:
22         tournée = ['Depot']; charge = 0; current = 'Depot'
23         while non_visites:
24             acc = [c for c in non_visites
25                    if charge + demandes[c] <= CAPACITE]
26             if not acc: break
27             voisin = min(acc, key=lambda c: distance(current,c))
28             tournée.append(voisin)
29             charge += demandes[voisin]
30             non_visites.remove(voisin)
31             current = voisin
32         tournée.append('Depot')
33         tournées.append((tournée, charge))
34     return tournées

```

Listing 3 – VRP – Heuristique greedy

Visualisation

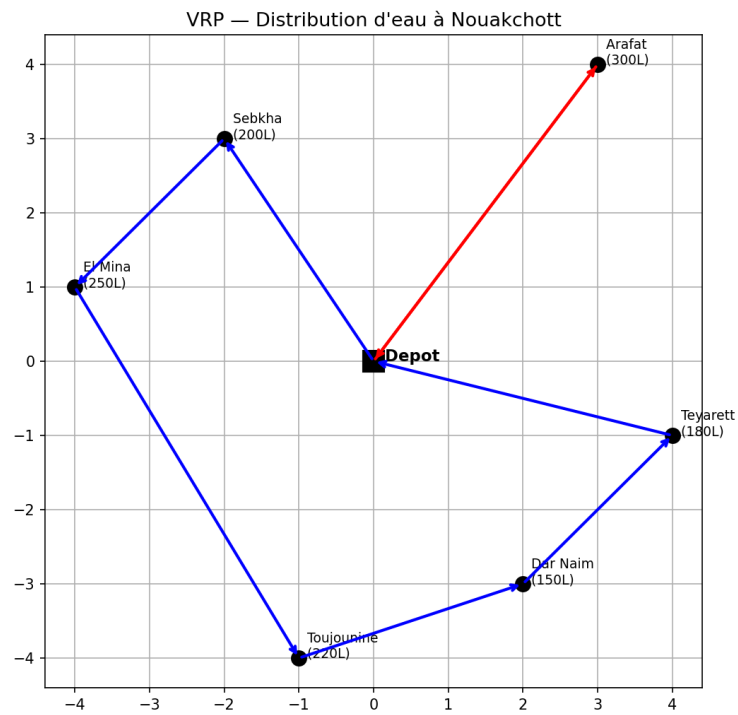


FIGURE 3 – Tournées de distribution d'eau à Nouakchott

Conclusion générale

Ce rapport a présenté trois applications de la Recherche Opérationnelle :

- **Le Simplexe** résout exactement les programmes linéaires. Il pivote sur le tableau jusqu'à atteindre l'optimum. PuLP automatise ce processus en Python.
- **Le TSP** est un problème NP-difficile. Le brute-force est exact mais exponentiel. Le greedy est rapide et donne de bonnes solutions sur de petites instances.
- **Le VRP** applique la logistique à un problème réel mauritanien. La contrainte de capacité impose plusieurs tournées. L'heuristique greedy donne une solution pratique et calculable rapidement.

Ces trois méthodes illustrent la complémentarité entre exactitude mathématique et efficacité algorithmique en Recherche Opérationnelle.